

**Laxmi Charitable Trust's**  
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**  
**Department of Information Technology (B.Sc.I.T Semester V)**  
**Artificial Intelligence Practical**  
**Practical – VIII**

|                             |                                  |
|-----------------------------|----------------------------------|
| Roll No.:T031               | Name:Ishita Patel                |
| Class:TYIT                  | Batch:1                          |
| Date of Assignment:22/08/26 | Date/Time of Submission:22/08/26 |

**1.Select any publicly available dataset (for example, Iris, Adult Income, or a classification dataset of your choice) and analyze it for possible bias and class imbalance.**

**Dataset: Titanic train.csv from Kaggle**

**1. Load and Explore the Dataset**

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# =====
```

```
# 1. Load and Explore the Dataset
```

```
# =====
```

```
df = pd.read_csv("train.csv")
```

```
print("First 5 Rows:")
```

```
print(df.head())
```

```
print("\nDataset Shape:")
```

```
print(df.shape)
```

```
print("\nColumn Names:")
```

```
print(df.columns.tolist())
```

```
print("\nDataset Information:")
```

```
print(df.info())
```

```
print("\nStatistical Summary:")
```

```
print(df.describe())
```

First 5 Rows:

|   | Loan_ID  | Gender | Married | Dependents | Education    | Self_Employed | \ |
|---|----------|--------|---------|------------|--------------|---------------|---|
| 0 | LP001002 | Male   | No      | 0          | Graduate     | No            |   |
| 1 | LP001003 | Male   | Yes     | 1          | Graduate     | No            |   |
| 2 | LP001005 | Male   | Yes     | 0          | Graduate     | Yes           |   |
| 3 | LP001006 | Male   | Yes     | 0          | Not Graduate | No            |   |
| 4 | LP001008 | Male   | No      | 0          | Graduate     | No            |   |

|   | ApplicantIncome | CoapplicantIncome | LoanAmount | Loan_Amount_Term | \ |
|---|-----------------|-------------------|------------|------------------|---|
| 0 | 5849            | 0.0               | NaN        | 360.0            |   |
| 1 | 4583            | 1508.0            | 128.0      | 360.0            |   |
| 2 | 3000            | 0.0               | 66.0       | 360.0            |   |
| 3 | 2583            | 2358.0            | 120.0      | 360.0            |   |
| 4 | 6000            | 0.0               | 141.0      | 360.0            |   |

|   | Credit_History | Property_Area | Loan_Status |
|---|----------------|---------------|-------------|
| 0 | 1.0            | Urban         | Y           |
| 1 | 1.0            | Rural         | N           |
| 2 | 1.0            | Urban         | Y           |
| 3 | 1.0            | Urban         | Y           |
| 4 | 1.0            | Urban         | Y           |

Dataset Shape:

(614, 13)

...

25% 1.000000

50% 1.000000

75% 1.000000

max 1.000000

## 2. Check Data Quality

```
# =====  
# 2. Check Data Quality  
# =====  
  
print("\nMissing Values:")  
print(df.isnull().sum())  
print("\nDuplicate Rows:")  
print(df.duplicated().sum())  
print("\nData Types:")  
print(df.dtypes)
```

```
Missing Values:  
Loan_ID          0  
Gender           13  
Married          3  
Dependents       15  
Education        0  
Self_Employed    32  
ApplicantIncome  0  
CoapplicantIncome 0  
LoanAmount       22  
Loan_Amount_Term 14  
Credit_History   50  
Property_Area    0  
Loan_Status      0  
dtype: int64  
  
Duplicate Rows:  
0  
  
Data Types:  
Loan_ID          str  
Gender           str  
Married          str  
Dependents       str  
...  
Credit_History   float64  
Property_Area    str  
Loan_Status      str  
dtype: object
```

## 3. Identify the Target Variable

```
# =====  
# 3. Identify the Target Variable  
# =====  
  
target = "Loan_Status"  
print("\nTarget Variable:", target)  
print("\nTarget Values:")  
print(df[target].value_counts())
```

```
Target Variable: Loan_Status  
  
Target Values:  
Loan_Status  
Y    422  
N    192  
Name: count, dtype: int64
```

#### 4. Analyze Class Distribution

```
# =====  
# 4. Analyze Class Distribution  
# =====  
class_distribution = df[target].value_counts()  
print("\nLoan Status Class Distribution:")  
print(class_distribution)  
print("\nClass Distribution Percentage:")  
print(df[target].value_counts(normalize=True) * 100)
```

```
Loan Status Class Distribution:  
Loan_Status  
Y      422  
N      192  
Name: count, dtype: int64  
  
Class Distribution Percentage:  
Loan_Status  
Y      68.729642  
N      31.270358  
Name: proportion, dtype: float64
```

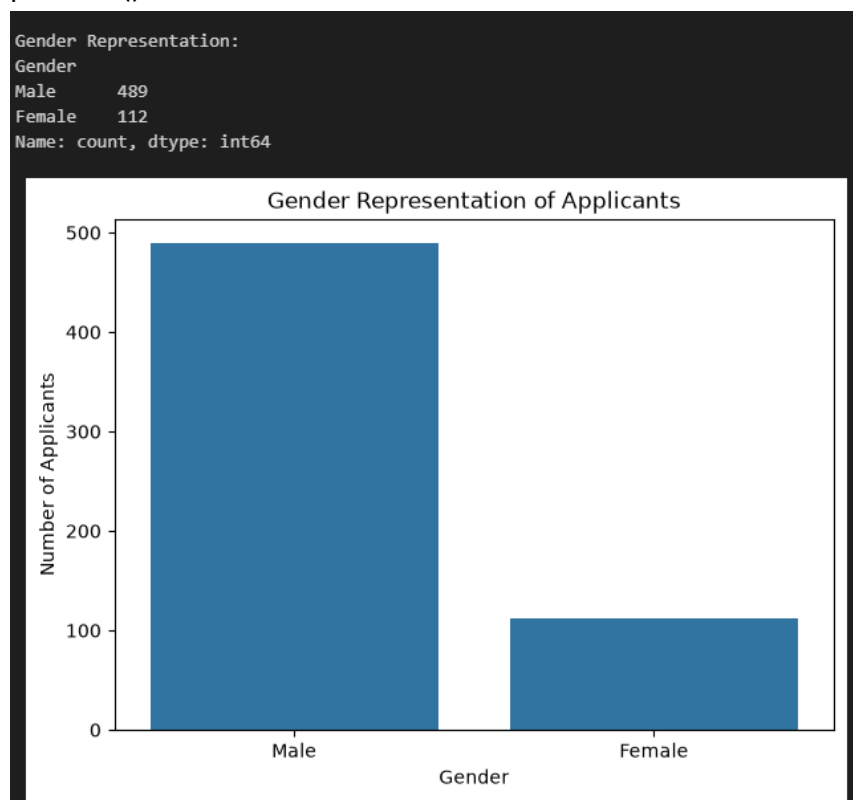
#### 5. Visualize Class Imbalance

```
# =====  
# 5. Visualize Class Imbalance  
# =====  
plt.figure(figsize=(7, 5))  
sns.countplot(x="Loan_Status", data=df)  
plt.title("Loan Approval Class Distribution")  
plt.xlabel("Loan Status")  
plt.ylabel("Number of Applicants")  
plt.show()
```



## 6. Analyze Gender Representation and display graph

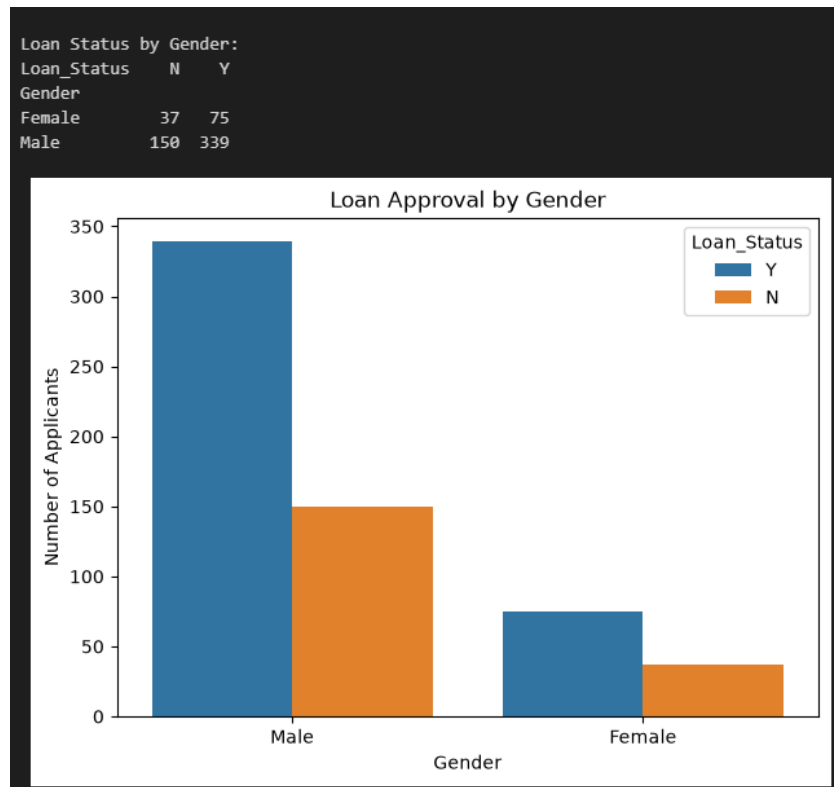
```
# =====  
# 6. Analyze Gender Representation and Display Graph  
# =====  
print("\nGender Representation:")  
print(df["Gender"].value_counts())  
plt.figure(figsize=(7, 5))  
sns.countplot(x="Gender", data=df)  
plt.title("Gender Representation of Applicants")  
plt.xlabel("Gender")  
plt.ylabel("Number of Applicants")  
plt.show()
```



## 7. Analyze Survival by Gender and display graph

```
# =====  
# 7. Analyze Loan Status by Gender  
# =====  
print("\nLoan Status by Gender:")  
print(pd.crosstab(df["Gender"], df["Loan_Status"]))  
plt.figure(figsize=(7, 5))  
sns.countplot(  
    x="Gender",  
    hue="Loan_Status",  
    data=df  
)  
plt.title("Loan Approval by Gender")  
plt.xlabel("Gender")  
plt.ylabel("Number of Applicants")
```

```
plt.show()
```



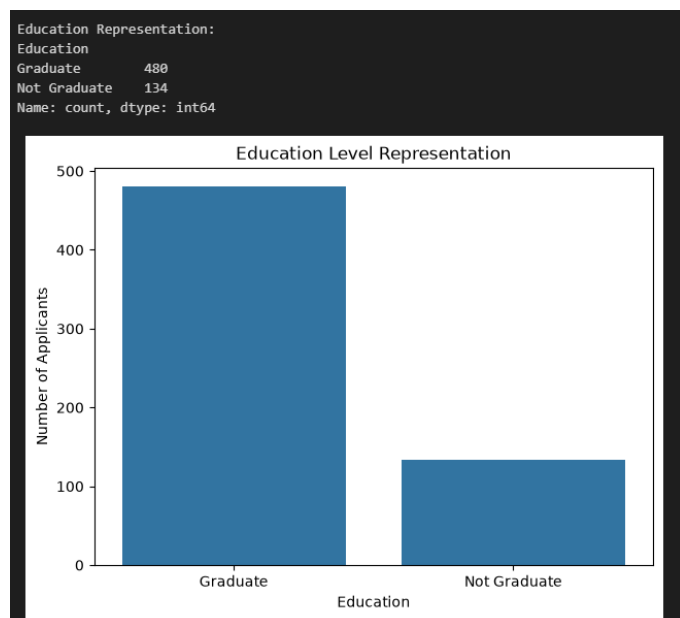
## 8. Analyze Passenger-Class Representation and display graph

```
# =====
```

```
# 8. Analyze Education Representation
```

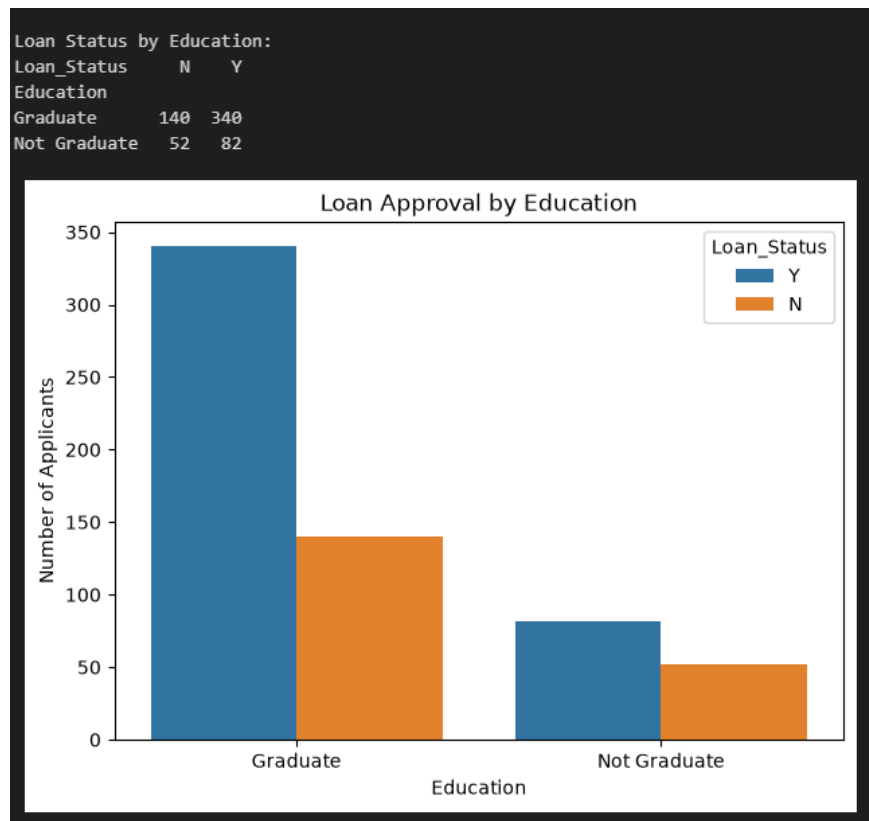
```
# =====
```

```
print("\nEducation Representation:")
print(df["Education"].value_counts())
plt.figure(figsize=(7, 5))
sns.countplot(x="Education", data=df)
plt.title("Education Level Representation")
plt.xlabel("Education")
plt.ylabel("Number of Applicants")
plt.show()
```



## 9. Analyze Survival by Passenger Class and display graph

```
# =====  
# 9. Analyze Loan Status by Education  
# =====  
print("\nLoan Status by Education:")  
print(pd.crosstab(df["Education"], df["Loan_Status"]))  
plt.figure(figsize=(7, 5))  
sns.countplot(  
    x="Education",  
    hue="Loan_Status",  
    data=df  
)  
plt.title("Loan Approval by Education")  
plt.xlabel("Education")  
plt.ylabel("Number of Applicants")  
plt.show()
```



## 10. Analyze Age Groups

```
# =====  
# 10. Analyze Applicant Income Groups  
# =====  
# Create income groups  
df["IncomeGroup"] = pd.cut(  
    df["ApplicantIncome"],  
    bins=[0, 2500, 5000, 10000, 1000000],  
    labels=[  
        "Low Income",  
        "Medium Income",
```

```

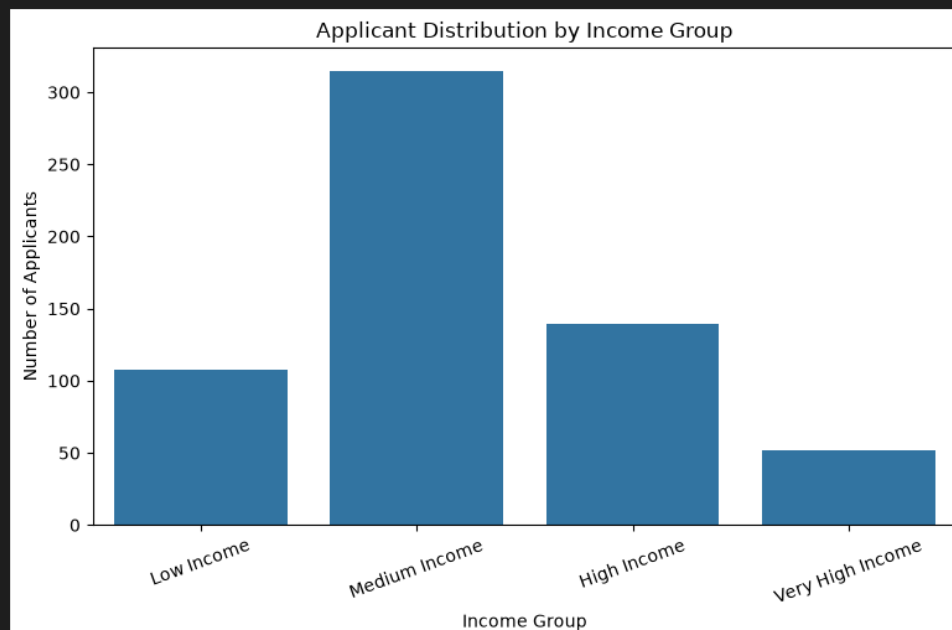
    "High Income",
    "Very High Income"
]
)
print("\nIncome Group Distribution:")
print(df["IncomeGroup"].value_counts().sort_index())
plt.figure(figsize=(9, 5))
sns.countplot(
    x="IncomeGroup",
    data=df
)
plt.title("Applicant Distribution by Income Group")
plt.xlabel("Income Group")
plt.ylabel("Number of Applicants")
plt.xticks(rotation=20)
plt.show()

```

```

Income Group Distribution:
IncomeGroup
Low Income      108
Medium Income   315
High Income     139
Very High Income  52
Name: count, dtype: int64

```



## 11. Analyze Survival by Age Group and display graph

```

# =====
# 11. Analyze Loan Status by Income Group
# =====
print("\nLoan Status by Income Group:")
print(
    pd.crosstab(
        df["IncomeGroup"],
        df["Loan_Status"]
    )
)

```

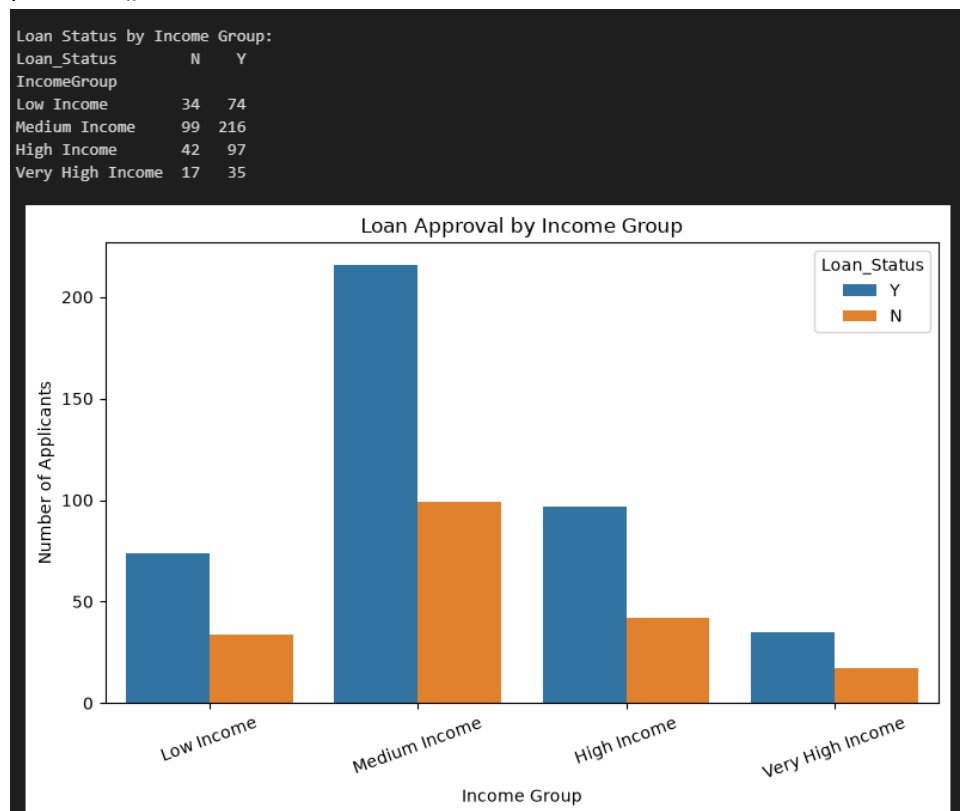
```

)
plt.figure(figsize=(9, 5))

sns.countplot(
    x="IncomeGroup",
    hue="Loan_Status",
    data=df
)

plt.title("Loan Approval by Income Group")
plt.xlabel("Income Group")
plt.ylabel("Number of Applicants")
plt.xticks(rotation=20)
plt.show()

```



## 12. Identify Possible Bias

```

# =====
# 12. Identify Possible Bias
# =====

print("\n===== POSSIBLE BIAS =====")

gender_bias = pd.crosstab(
    df["Gender"],
    df["Loan_Status"],
    normalize="index"
) * 100

print("\nLoan Approval Percentage by Gender:")
print(gender_bias)

education_bias = pd.crosstab(
    df["Education"],

```

```

df["Loan_Status"],
    normalize="index"
) * 100
print("\nLoan Approval Percentage by Education:")
print(education_bias)
property_bias = pd.crosstab(
    df["Property_Area"],
    df["Loan_Status"],
    normalize="index"
) * 100
print("\nLoan Approval Percentage by Property Area:")
print(property_bias)
print("""

```

```

===== POSSIBLE BIAS =====

Loan Approval Percentage by Gender:
Loan_Status      N      Y
Gender
Female      33.035714  66.964286
Male       30.674847  69.325153

Loan Approval Percentage by Education:
Loan_Status      N      Y
Education
Graduate      29.166667  70.833333
Not Graduate  38.805970  61.194030

Loan Approval Percentage by Property Area:
Loan_Status      N      Y
Property_Area
Rural      38.547486  61.452514
Semiurban  23.175966  76.824034
Urban      34.158416  65.841584

```

### 13. Impact on ML

```
# =====
```

```
# 13. Impact on Machine Learning
```

```
# =====
```

1. Class imbalance can make the model favor the majority class.
2. Missing values can reduce model performance.
3. Gender and other demographic variables may introduce bias.
4. Credit history can have a strong influence on predictions.
5. The model may learn existing patterns or bias from historical data.
6. Accuracy alone may not be enough to evaluate fairness.

### 14. Suggested solutions

```
# =====
```

```
# 14. Suggested Solutions
```

```
# =====
```

1. Handle missing values using suitable imputation methods.
2. Encode categorical variables into numerical values.
3. Use stratified train-test splitting.

4. Handle class imbalance using class weights or resampling.
5. Use Precision, Recall, F1-score and Confusion Matrix.
6. Compare model performance across different demographic groups.
7. Check for unfair differences in approval rates.
8. Remove unnecessary features if they introduce unwanted bias.

## 15. Final conclusion

# =====

# 15. Final Conclusion

# =====

The Loan Eligibility dataset contains information about applicants, their income, education, credit history, property area and loan status.

The analysis helps identify class distribution, demographic representation and possible bias in loan approval decisions.

Factors such as credit history, income, education and property area may have an important relationship with loan approval.

Missing values, class imbalance and demographic differences should be handled carefully before developing a machine learning model.

Therefore, proper preprocessing, balanced training data and fairness evaluation are important for building a reliable and unbiased loan eligibility prediction system.

## Dataset: German Credit Risk Dataset from Kaggle

### 1. Load and Explore the Dataset

```
# =====  
# GERMAN CREDIT RISK DATASET - BIAS ANALYSIS  
# =====  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
# =====  
# 1. Load and Explore the Dataset  
# =====  
df = pd.read_csv("German_Credit_Risk.csv")  
print("First 5 Rows:")  
print(df.head())  
print("\nDataset Shape:")  
print(df.shape)  
print("\nColumn Names:")  
print(df.columns.tolist())  
print("\nDataset Information:")  
df.info()  
print("\nStatistical Summary:")  
print(df.describe(include="all"))
```

```
First 5 Rows:  
   Age  Sex  Job  Housing  Saving  accounts  Checking  account  Credit amount  \  
0    67  male    2    own          little          little      1169  
1    22  female  2    own          little      moderate      5951  
2    49  male    1    own          little          little      2096  
3    45  male    2    free          little          little      7882  
4    53  male    2    free          little          little      4870  
  
   Duration  Purpose  Risk  
0         6      radio/TV  good  
1        48      radio/TV  bad  
2        12    education  good  
3        42  furniture/equipment  good  
4        24          car  bad  
  
Dataset Shape:  
(20, 10)  
  
Column Names:  
['Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account', 'Credit amount', 'Duration', 'Purpose', 'Risk']  
  
Dataset Information:  
<class 'pandas.DataFrame'>  
RangeIndex: 20 entries, 0 to 19  
Data columns (total 10 columns):  
...  
25%      1355.750000  12.000000   NaN   NaN  
50%      2534.500000  24.000000   NaN   NaN  
75%      5413.250000  31.500000   NaN   NaN  
max       9055.000000  48.000000   NaN   NaN
```

### 2. Check Data Quality

```
# =====  
# 2. Check Data Quality  
# =====  
print("\n===== DATA QUALITY =====")  
print("\nMissing Values:")
```

```
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
print("\nData Types:")
print(df.dtypes)
```

```
===== DATA QUALITY =====

Missing Values:
Age                0
Sex                0
Job                0
Housing            0
Saving accounts    0
Checking account   0
Credit amount     0
Duration           0
Purpose            0
Risk               0
dtype: int64

Duplicate Rows:
0

Data Types:
Age                int64
Sex                str
Job                int64
Housing            str
Saving accounts    str
...
Duration           int64
Purpose            str
Risk               str
dtype: object
```

### 3. Identify the Target Variable

```
# =====
# 3. Identify the Target Variable
# =====

target = "Risk"
print("\n===== TARGET VARIABLE =====")
print("Target Variable:", target)
print("\nTarget Values:")
print(df[target].value_counts())
```

```
===== TARGET VARIABLE =====
Target Variable: Risk

Target Values:
Risk
good    14
bad      6
Name: count, dtype: int64
```

### 4. Analyze Class Distribution

```
# =====
# 4. Analyze Class Distribution
```

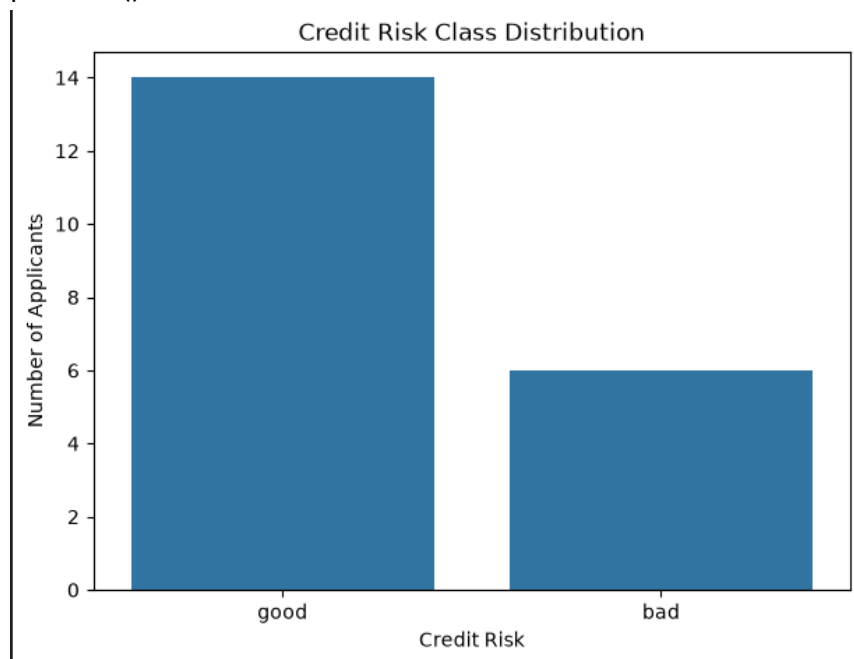
```
# =====
print("\n===== CLASS DISTRIBUTION =====")
class_distribution = df["Risk"].value_counts()
print(class_distribution)
print("\nClass Distribution Percentage:")
print(df["Risk"].value_counts(normalize=True) * 100)
```

```
===== CLASS DISTRIBUTION =====
Risk
good      14
bad        6
Name: count, dtype: int64

Class Distribution Percentage:
Risk
good      70.0
bad       30.0
Name: proportion, dtype: float64
```

## 5. Visualize Class Imbalance

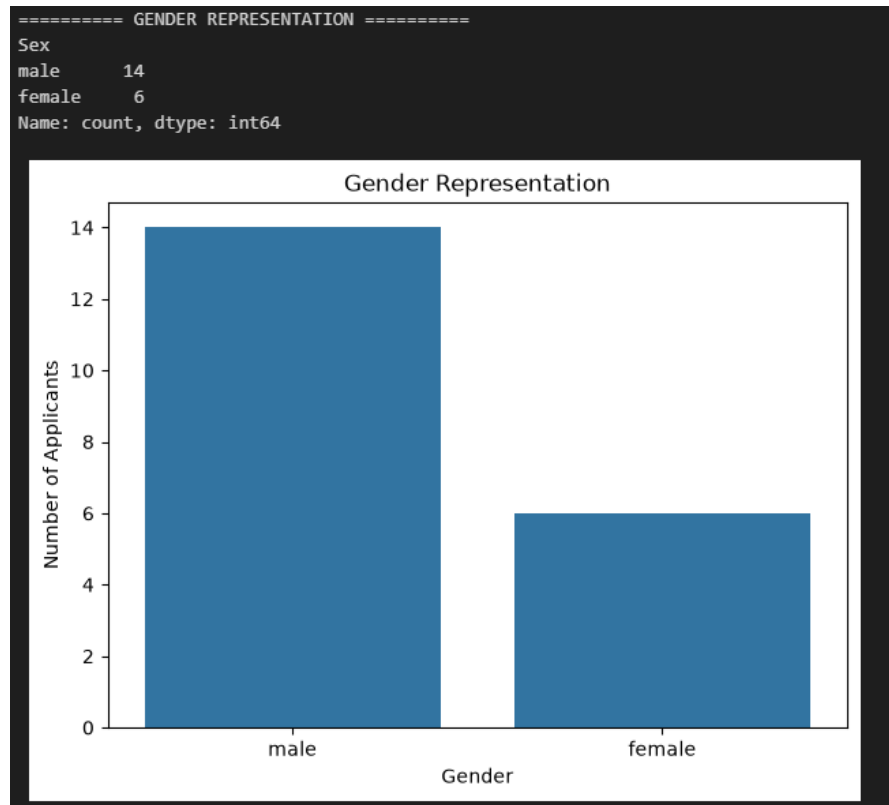
```
# =====
# 5. Visualize Class Imbalance
# =====
plt.figure(figsize=(7, 5))
sns.countplot(x="Risk", data=df)
plt.title("Credit Risk Class Distribution")
plt.xlabel("Credit Risk")
plt.ylabel("Number of Applicants")
plt.show()
```



## 6. Analyze Gender Representation and display graph

```
# =====
# 6. Analyze Gender Representation and Display Graph
# =====
print("\n===== GENDER REPRESENTATION =====")
```

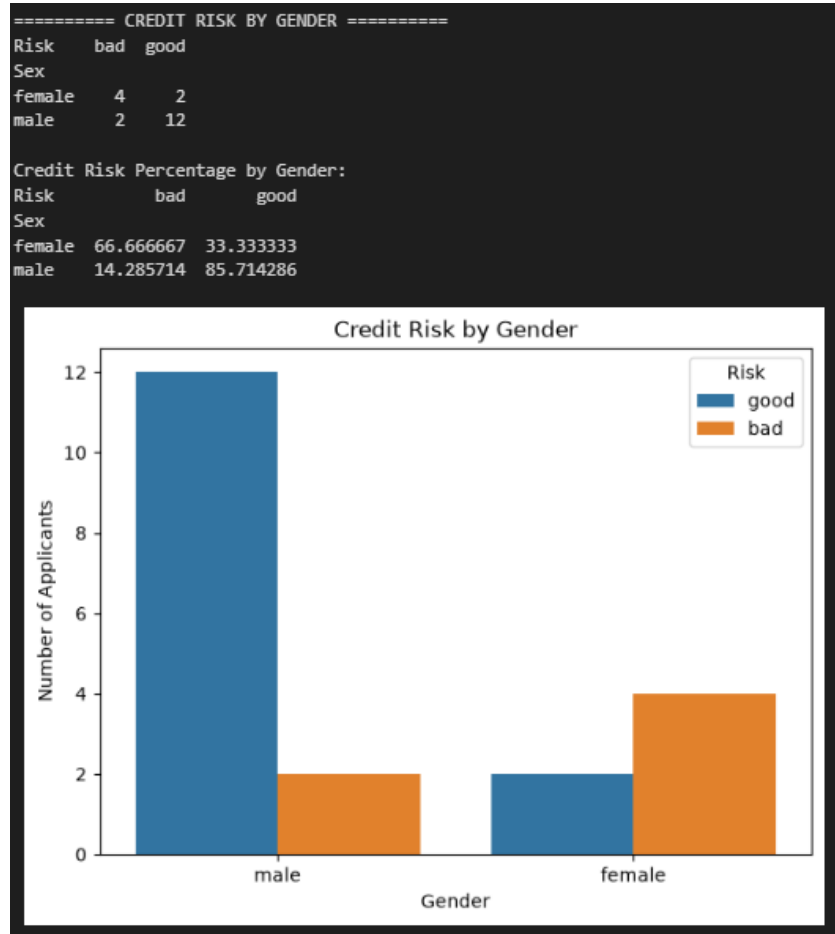
```
print(df["Sex"].value_counts())
plt.figure(figsize=(7, 5))
sns.countplot(x="Sex", data=df)
plt.title("Gender Representation")
plt.xlabel("Gender")
plt.ylabel("Number of Applicants")
plt.show()
```



## 7. Analyze Credit Risk by Gender and display graph

```
# =====
# 7. Analyze Credit Risk by Gender and Display Graph
# =====
print("\n===== CREDIT RISK BY GENDER =====")
print(pd.crosstab(df["Sex"], df["Risk"]))
print("\nCredit Risk Percentage by Gender:")
gender_risk = pd.crosstab(
    df["Sex"],
    df["Risk"],
    normalize="index"
) * 100
print(gender_risk)
plt.figure(figsize=(7, 5))
sns.countplot(
    x="Sex",
    hue="Risk",
    data=df
)
plt.title("Credit Risk by Gender")
```

```
plt.xlabel("Gender")
plt.ylabel("Number of Applicants")
plt.show()
```



## 8. Analyze Age Groups and display graph

```
# =====
# 8. Analyze Age Groups and Display Graph
# =====

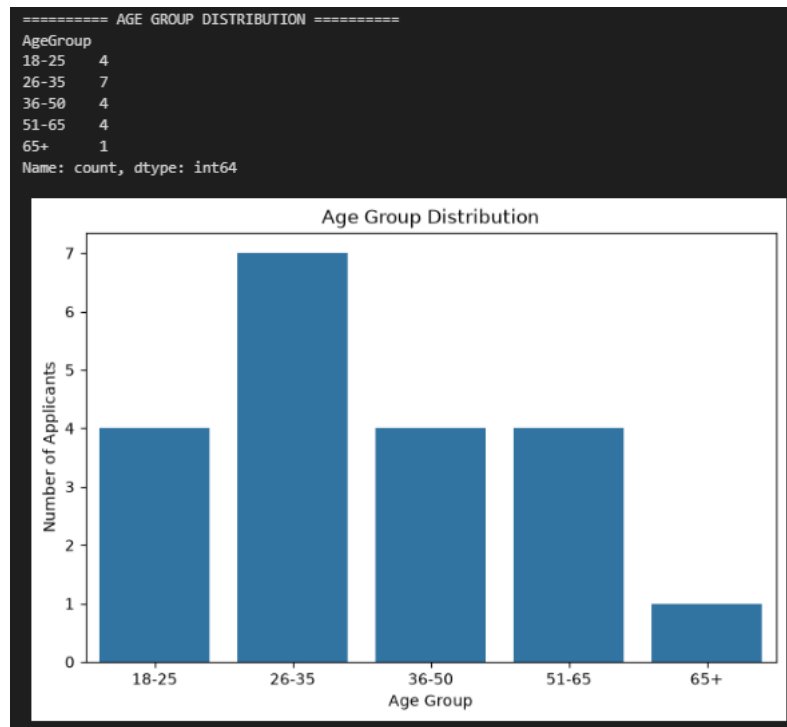
# Create Age Groups
df["AgeGroup"] = pd.cut(
    df["Age"],
    bins=[18, 25, 35, 50, 65, 100],
    labels=[
        "18-25",
        "26-35",
        "36-50",
        "51-65",
        "65+"
    ]
)

print("\n===== AGE GROUP DISTRIBUTION =====")
print(df["AgeGroup"].value_counts().sort_index())
plt.figure(figsize=(8, 5))
sns.countplot(
    x="AgeGroup",
```

```

data=df
)
plt.title("Age Group Distribution")
plt.xlabel("Age Group")
plt.ylabel("Number of Applicants")
plt.show()

```



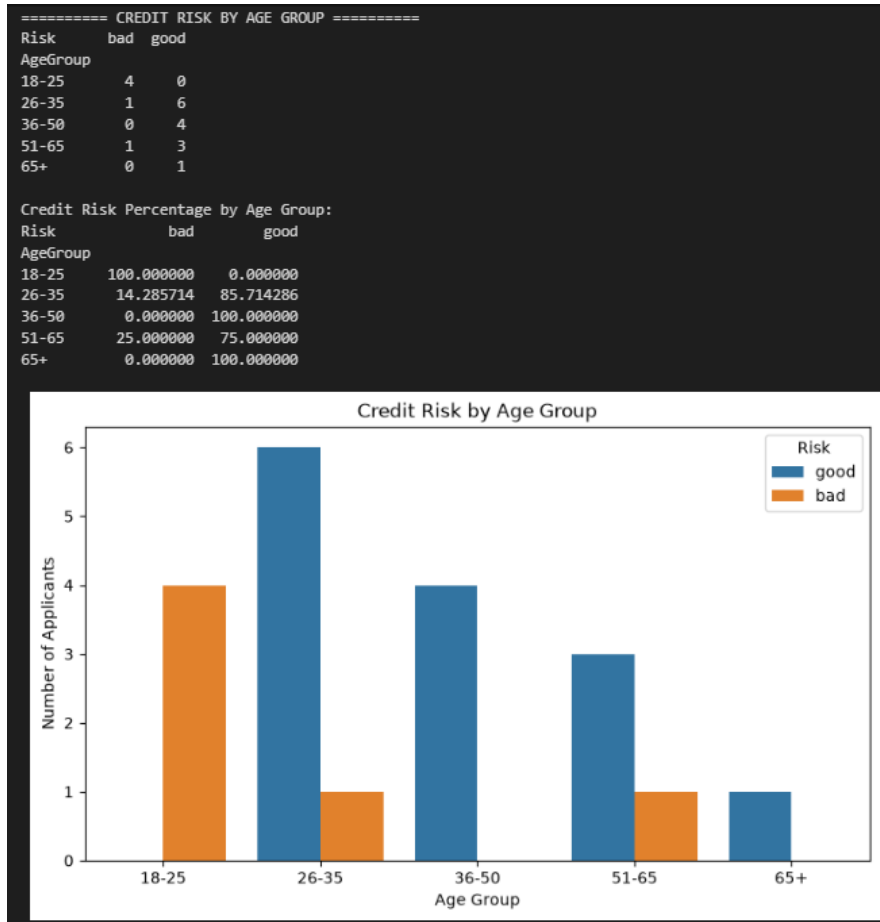
### 9. Analyze Credit Risk by Age Group and display graph

```

# =====
# 9. Analyze Credit Risk by Age Group and Display Graph
# =====
print("\n===== CREDIT RISK BY AGE GROUP =====")
print(
    pd.crosstab(
        df["AgeGroup"],
        df["Risk"]
    )
)
print("\nCredit Risk Percentage by Age Group:")
age_risk = pd.crosstab(
    df["AgeGroup"],
    df["Risk"],
    normalize="index"
) * 100
print(age_risk)
plt.figure(figsize=(9, 5))
sns.countplot(
    x="AgeGroup",
    hue="Risk",
    data=df
)

```

```
)
plt.title("Credit Risk by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Applicants")
plt.show()
```



## 10. Analyze Housing Category Representation and display graph

```
# =====
# 10. Analyze Housing Category Representation
# and Display Graph
# =====

print("\n===== HOUSING REPRESENTATION =====")
print(df["Housing"].value_counts())
plt.figure(figsize=(7, 5))
sns.countplot(
    x="Housing",
    data=df
)
plt.title("Housing Category Representation")
plt.xlabel("Housing Category")
plt.ylabel("Number of Applicants")
plt.show()
```

```
===== HOUSING REPRESENTATION =====
```

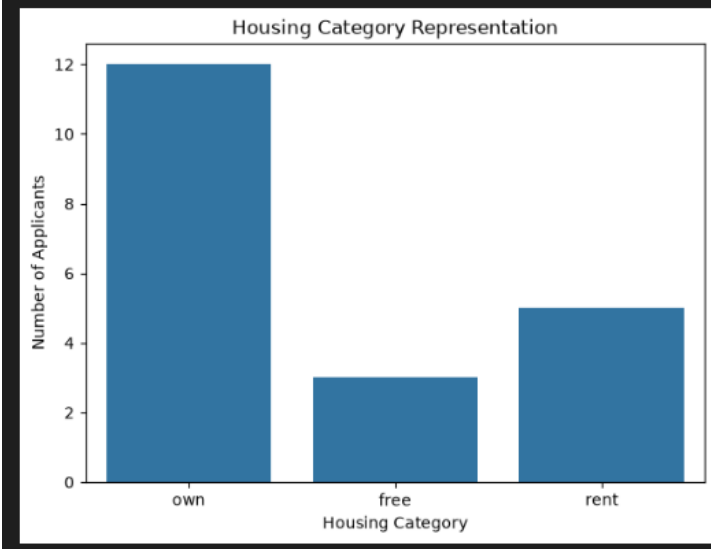
```
Housing
```

```
own    12
```

```
rent    5
```

```
free     3
```

```
Name: count, dtype: int64
```



### 11. Analyze Credit Risk by Housing Category and display graph

```
# =====
```

```
# 11. Analyze Credit Risk by Housing Category
```

```
# and Display Graph
```

```
# =====
```

```
print("\n===== CREDIT RISK BY HOUSING =====")
```

```
print(
```

```
    pd.crosstab(
```

```
        df["Housing"],
```

```
        df["Risk"]
```

```
    )
```

```
)
```

```
print("\nCredit Risk Percentage by Housing:")
```

```
housing_risk = pd.crosstab(
```

```
    df["Housing"],
```

```
    df["Risk"],
```

```
    normalize="index"
```

```
) * 100
```

```
print(housing_risk)
```

```
plt.figure(figsize=(8, 5))
```

```
sns.countplot(
```

```
    x="Housing",
```

```
    hue="Risk",
```

```
    data=df
```

```
)
```

```
plt.title("Credit Risk by Housing Category")
```

```
plt.xlabel("Housing Category")
```

```
plt.ylabel("Number of Applicants")
```

```
plt.show()
```

# ===== CREDIT RISK BY HOUSING =====

Risk      bad    good

Housing

free      1      2

own      2      10

rent      3      2

Credit Risk Percentage by Housing:

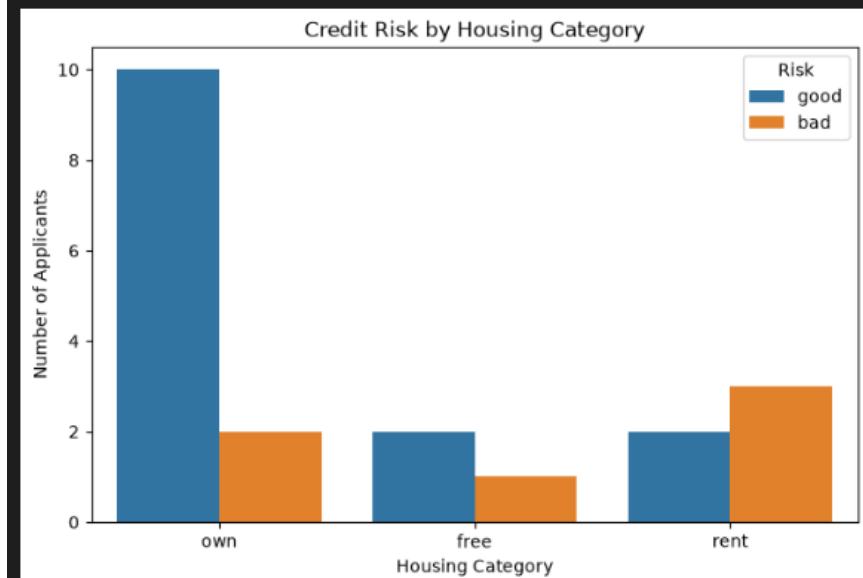
Risk      bad      good

Housing

free      33.333333    66.666667

own      16.666667    83.333333

rent      60.000000    40.000000



## 12. Identify Possible Bias

Possible Bias Identified:

1. Gender:

Credit risk distribution may differ between males and females.

2. Age:

Different age groups may have different credit risk levels.

3. Housing:

Credit risk may vary among people who own, rent or have free housing.

4. Class Imbalance:

One Risk class may contain significantly more records than the other class.

5. Historical Bias:

The dataset contains historical credit decisions, so existing social or financial patterns may be reflected in the data.

## 13. Impact on ML

1. Class imbalance can cause the model to favor the majority class.

2. Gender-related patterns may introduce demographic bias.

3. Age-related patterns may cause unequal predictions for different age groups.

4. Housing information may indirectly represent socioeconomic status.

5. The model may learn historical biases present in the dataset.

6. Accuracy alone may not be enough to evaluate the model.

7. A model can have high overall accuracy but perform poorly for a particular demographic group.

#### **14. Suggested Solutions**

1. Handle class imbalance using class weights or resampling.
2. Check and handle missing values before model training.
3. Encode categorical variables properly.
4. Use stratified train-test splitting.
5. Evaluate the model using Accuracy, Precision, Recall and F1-score.
6. Use a Confusion Matrix to understand prediction errors.
7. Compare model performance across gender and age groups.
8. Check whether housing information introduces unwanted bias.
9. Apply fairness evaluation before deploying the model.

#### **15. Final Conclusion**

The German Credit Risk dataset contains information about the age, gender, employment, housing, financial accounts, credit amount, duration, purpose and credit risk of applicants.

The analysis shows that credit risk can vary across different gender groups, age groups and housing categories.

The dataset may also contain class imbalance and historical patterns that can introduce bias into machine learning predictions.

Therefore, proper data preprocessing, class balancing, suitable evaluation metrics and fairness analysis are important before building a machine learning model using this dataset.

A reliable credit risk prediction system should provide accurate predictions while also reducing unfair treatment of different demographic groups.